

Homework 2

Name: Fong Yu Lin, PID: A18496379

1 Automatic Differentiation

Problem A: Backpropagation and Weight Initialization

i. Backpropagation Algorithm and ReLU

Solution A.i: Say the neural network with all weights equal 0 is network 2, and the other neural network is network 1. Network 1 successfully learned to separated the two classes very quickly after starting, while network 2 doesn't change. Network 1 learns much faster because random initialization breaks symmetry between hidden units. Different hidden units can learn different features, so backpropagation gives useful and different updates to the weights.

Network 2 doesn't learn well because zero initialization makes the hidden units symmetric. If all weights start at zero, then the hidden units in the same layer get the same inputs and the same gradients, so they keep learning the same function. In addition, with ReLU activation,

$$\text{ReLU}(z) = \max(0, z)$$

, the gradient is zero when $z \leq 0$. So, if a ReLU unit outputs zero, little or no gradient can flow backward through that unit. This stops many weights from being updated effectively.

ii. Backpropagation Algorithm and Sigmoid

Solution A.ii: The learning speed of Sigmoid is a lot slower compared to ReLU function. For network 1, while ReLU converges within 200 epoches, Sigmoid uses about 900 epoches. As for network 2, interestingly, the weights starting to change a little by little and not only stuck in 0 anymore. Although the changes are slow and small, they are indeed changing. However, it cannot separate the two classes within 4000 epoch.

When the activation function is changed to sigmoid, both networks learn more slowly than in

the ReLU case. This is because the sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

has small gradients when z is very positive or very negative. Its derivative is

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)),$$

which becomes close to zero when the sigmoid saturates near 0 or 1. This makes backpropagation slower because the gradients passed to earlier layers become smaller.

For the network 1, the model can still learn, but it converges more slowly than the ReLU network. For network 2, the weights change a little because

$$\sigma(0) = 0.5$$

and the sigmoid derivative at zero is not zero. However, zero initialization still has the symmetry problem: hidden units in the same layer receive the same gradients and tend to learn the same features.

Therefore, the network has much less effective capacity and does not separate the two classes well even after many iterations.

Problem B: Data Shuffling

Solution B: If we train a ReLU network using SGD and loop through all the negative examples before any positive examples, the updates are highly biased toward only one class at the beginning of training. This can push the pre-activation values of some ReLU neurons below zero for many inputs. Since ReLU has zero gradient when its input is negative,

$$\frac{d}{dz}\text{ReLU}(z) = 0 \quad \text{when } z < 0,$$

those neurons may stop receiving useful gradient updates. This is the dying ReLU problem.

Once many ReLU units become inactive, later positive examples may not be able to reactivate them because no gradient flows through those dead units. As a result, the network may learn slowly or get stuck with poor performance. Shuffling the data helps avoid this by mixing positive and negative examples, so the SGD updates are less biased and the ReLU units are less likely to die early.

2 Convolutional Neural Networks

Problem A: Convolution

i. Parameter counts

Solution A.i:

Parameter Count:

$$(k_w \times k_h \times C_{in} + b) \times C_{out} = (5 \times 5 \times 3 + 1) \times 16 = 1216$$

ii. Shape of the Output Tensor

Solution A.ii: *Output Size:*

$$\lfloor \frac{n + 2p - k}{s} \rfloor + 1 = \lfloor \frac{32 + 2(0) - 5}{1} \rfloor + 1 = 28$$

$$28 \times 28 \times 16$$

Problem B: Padding

Solution B: *Benefits of zero-padding is we can have control over the output size, keep the edge information, and prevent the feature map to shrink too quickly. Drawback is that it has some zero values that are not part of the original image. Convolutions near the border may therefore use fake information, and leads to the features near the edge less reliable.*

Problem C: Pooling

i. Average Pooling

Solution C.i:

$$\begin{bmatrix} 1 & 0.5 \\ 0.5 & 0.25 \end{bmatrix}, \begin{bmatrix} 0.5 & 1 \\ 0.25 & 0.5 \end{bmatrix}, \begin{bmatrix} 0.25 & 0.5 \\ 0.5 & 1 \end{bmatrix}, \begin{bmatrix} 0.5 & 0.25 \\ 1 & 0.5 \end{bmatrix}$$

ii. Max Pooling

Solution C.ii:

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

Problem D: Pytorch implementation

i. PyTorch Notebook Implementation

Solution D.i:

I implemented a convolutional neural network using PyTorch's `nn.Sequential`. The model uses two convolutional blocks followed by fully connected layers. Each convolutional block contains convolutional layers, batch normalization, ReLU activation, max pooling, and dropout. The model has fewer than one million parameters.

The dropout probabilities were tested using

$$p = 0.0, 0.1, 0.2, \dots, 0.9$$

Each model was trained for one epoch with batch size 32, and the best dropout probability was then used to train the final model for 10 epochs.

<i>Dropout Probability</i>	<i>Test Accuracy After 1 Epoch</i>
0.0	0.9778
0.1	0.9869
0.2	0.9867
0.3	0.9858
0.4	0.9843
0.5	0.9829
0.6	0.9761
0.7	0.9688
0.8	0.9175
0.9	0.1135

The best one-epoch dropout probability was $p = 0.1$. Using this dropout probability, the final model was trained for 10 epochs and achieved a final test accuracy of 0.9929.

ii. Most Effective Strategy

Solution D.ii: *After several tries of different strategies, the most effective strategy I found is using multiple convolutional layers before pooling. I'm also using batch normalization after convolutional layers, and keeping the number of channels moderate so that the model stayed under the one-million-parameter limit. Stacking two convolutional layers before each max-pooling layer makes the model to learn more useful local features before reducing spatial resolution.*

As for to the regularization methods, batch normalization was the most consistently useful because it improved optimization speed and made training more stable. Dropout helped when the dropout probability was moderate, but large dropout probabilities hurt performance because too many activations were removed during training. Weight decay also helped slightly, but its effect was smaller than batch normalization in this experiment.

In my dropout sweep, $p = 0.1$ achieved the best one-epoch test accuracy, so I used $p = 0.1$ for the final 10-epoch model.

iii. Foresee Problems

Solution D.iii: *Yes, there is a problem with this way of validating our hyperparameters. We are using the test set to choose the best dropout probability, which means the test set is no longer an unbiased estimate of final generalization performance. If we evaluate many hyperparameter settings on the test set and choose the best one, we are indirectly overfitting to the test set.*

To improve this, we can split the original training set into a training set and a validation set. The validation set should be used to choose hyperparameters such as dropout probability and weight decay. The test set should only be used once at the end to report the final performance of the selected model.

3 Sequence Processing

Problem A:

i.

Solution A.i:

Let $x_t = [a_t, b_t]^T$, and let c_{t-1} be the carry from the previous time step.

Say

$$s_t = a_t + b_t + c_{t-1}$$

Meaning add up the two inputs: a_t and b_t and add the carry (c_{t-1}) from the previous.

3 hidden units can be:

$$h_{1,t} = 1 \text{ if } s_t \geq 1$$

$$h_{2,t} = 1 \text{ if } s_t \geq 2$$

$$h_{3,t} = 1 \text{ if } s_t \geq 3$$

The second hidden unit stores the carry: $c_t = h_{2,t}$

ii.

Solution A.ii:

$$U = \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad W = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \quad b_h = \begin{bmatrix} -0.5 \\ -1.5 \\ -2.5 \end{bmatrix}$$

For the output layer:

$$V = \begin{bmatrix} 1 & -1 & 1 \end{bmatrix}, \quad b_y = -0.5$$

This gives output 1 when $s_t = 1$ or $s_t = 3$, and output 0 when $s_t = 0$ or $s_t = 2$, which matches binary addition.

iii.

Solution A.iii: Use the example given in the problem. The two inputs are

$$a = 1, 1, 1, 0, 0, 1, 0$$

$$b = 0, 1, 0, 0, 1, 1, 0.$$

Say the initial hidden state is

$$h_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

so the initial carry is $c_0 = 0$. At each time step, we have

$$s_t = a_t + b_t + c_{t-1}.$$

The output bit is 1 when $s_t = 1$ or $s_t = 3$, and 0 when $s_t = 0$ or $s_t = 2$. The new carry is

$$c_t = h_{2,t}$$

t	a_t	b_t	c_{t-1}	s_t	y_t	c_t
1	1	0	0	1	1	0
2	1	1	0	2	0	1
3	1	0	1	2	0	1
4	0	0	1	1	1	0
5	0	1	0	1	1	0
6	1	1	0	2	0	1
7	0	0	1	1	1	0

Therefore, we get:

$$1, 0, 0, 1, 1, 0, 1,$$

Thus, the construction correctly implements the binary addition.